GenAI-Powered Postsecondary Computer Science Education Tools: A Systematic Literature Review

Owen Sizemore
Duke University
Durham, NC, USA
owen.sizemore@duke.edu

Seth Bernstein
University of Michigan
Ann Arbor, MI, USA
sethbern@umich.edu

Audrey Ostrom
University of California, Santa Cruz
Santa Cruz, CA, USA
aostrom@ucsc.edu

Thomas Rexin
North Carolina State University
Raleigh, NC, USA
tjrexin@ncsu.edu

Kristin Stephens-Martinez
Duke University
Durham, NC, USA
ksm@cs.duke.edu

Abstract

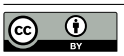
The emergence of commercial Generative AI (GenAI) tools like ChatGPT has led to a large body of research investigating how these tools are used in postsecondary CS education (CSEd) and their impact on student learning. Instructors and researchers have built new tools that leverage the powerful capabilities of GenAI, providing a pedagogically-oriented alternative to commercial tools. This emerging phenomenon has created a large ecosystem of GenAI-powered CSEd tools, but there is a limited understanding of how these tools are developed, how they utilize existing knowledge of education and learning, and how they are evaluated. To answer these questions, we conducted a systematic literature review on GenAI-powered tools in postsecondary CSEd, screening 1,834 search results from ACM Digital Library, IEEE Xplore, Springer Link, and Scopus, resulting in 72 papers for analysis. Through inductive coding, we found that tool development is strongly motivated by desires to provide personalized learning experiences for students and to address the potential risks of commercial LLMs. Further, we found that tool development was often grounded in educational framings like scaffolding and cognitive load theory. Tools frequently included features to avoid giving direct answers to students and adapt to their responses over time. In terms of evaluation, the most common methods include post-surveys and logging usage data. We close with a discussion of broader trends we observed across the papers and offer considerations for improving the development of GenAI-powered CSEd tools in the future.

CCS Concepts

• **Social and professional topics → Computing education**.

Keywords

systematic literature review, computer science education, generative AI, tools, tutors



This work is licensed under a Creative Commons Attribution 4.0 International License.
Koli Calling '26, Koli, Finland
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2752-8/2026/11
https://doi.org/10.1145/3856208.3856224

ACM Reference Format:
Owen Sizemore, Seth Bernstein, Audrey Ostrom, Thomas Rexin, and Kristin Stephens-Martinez. 2026. GenAI-Powered Postsecondary Computer Science Education Tools: A Systematic Literature Review. In *26th Koli Calling International Conference on Computing Education Research (Koli Calling '26), November 05–08, 2026, Koli, Finland*. ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3856208.3856224

1 Introduction

The rapid adoption of commercial Generative AI (GenAI) tools like ChatGPT [73], Claude [5], and Microsoft Copilot [68] has resulted in a large body of research investigating how these tools are used in the context of postsecondary computer science education (CSEd) and their impact on student learning [10, 12, 22, 41, 44, 45, 60, 79, 80]. There are several competing interests when it comes to adopting GenAI in CS coursework [78]: while students report valuing the problem-solving guidance [22], personalized explanations and examples [41], and timely, always-available feedback [22, 79] from GenAI, instructors want to ensure they are preparing students to evaluate and debug AI-generated code and leverage GenAI in advanced CS work [41, 47, 66, 78, 104]. Nonetheless, students and instructors share several perspectives on GenAI, including concerns about overreliance [34, 104] and impacts on independent problem-solving, metacognitive, and critical thinking skills [41, 93].

The lack of consensus surrounding how to approach commercial GenAI tools in postsecondary CS coursework [28, 55, 78, 79] has challenged instructors to develop their own strategies for regulating student-AI interactions. Some instructors have opted to ban the use of commercial tools in their courses, but critics argue that this strategy is not feasible given the difficulty of enforcing a ban and the potential to stir students' curiosity in the tools [55]. Others have investigated the creation of customized tools that extend the capabilities of commercial tools through features that align the technology with course learning objectives or pedagogical theories [9, 46, 61, 62]. There is growing interest in developing GenAI-powered CSEd tools as an education-minded alternative to commercial tools, as they have the potential to deliver the benefits of GenAI while avoiding problematic learning pitfalls [23].

The GenAI-powered CSEd tool ecosystem spans a large group of tools with distinct architectures, user contexts, and pedagogical features [79], creating challenges for instructors seeking practical

guidance on how these tools could be leveraged in their own coursework. The lack of centralized information on what tools currently exist, the approaches taken to develop them, and how they have been evaluated in experimental settings or interventions remains unclear, limiting our wider understanding of learning-oriented alternatives to commercial GenAI tools. This work narrows this research gap by conducting a systematic literature review (SLR) of GenAI-powered postsecondary CSEd tools, classifying tool characteristics and analyzing broader trends and practices in the space. We ask the following research questions:

(RQ1) What approaches are currently used to build GenAI-powered postsecondary CSEd tools, and how are they motivated?
(RQ2) What educational theories and framings inform the development of GenAI-powered postsecondary CSEd tools?
(RQ3) What methods are used to empirically evaluate the effects of GenAI-powered postsecondary CSEd tools?

2 Related Work

2.1 AI in Computer Science Education

Before the proliferation of GenAI, intelligent tutoring systems (ITSes) were a popular approach for providing scalable, individualized programming support, often leveraging classical AI techniques such as Bayesian networks, case-based reasoning, fuzzy sets and rules, and genetic algorithms [29]. Crow *et al.* [20] reviewed ITSes for programming education and cataloged the pedagogical strategies these systems used to support learners. Le *et al.* [56] classified the tutoring approaches ITSes took, spanning the use of examples, simulations, dialogue, program analysis, feedback, and collaboration. Létourneau *et al.* [57] found that ITSes customizing instruction to a student's needs or providing immediate, personalized feedback improved learning outcomes.

Literature reviews assessing the opportunities and challenges of integrating GenAI into CSEd emerged following the rise of commercial tools [2, 11, 12, 16, 79]. Prather *et al.* [79] conducted one of the broadest examinations of GenAI in CSEd, contextualizing the results of a systematic literature review with surveys of instructors and software developers to capture trends in GenAI research, teaching practices, and tools. They found that relative to unstructured commercial tool access, guidance on how to use GenAI and instructor-provided guardrails improved student learning outcomes, such as code comprehension and problem-solving performance. They also found that 35% of computing educators incorporated GenAI into their course at the time of the survey, which contrasted the higher adoption rates among professional software developers [60]. Prather *et al.* distinguish "general-purpose" GenAI tools from tools with "instructor guardrails" in their literature review, but they did not examine how tools were designed, what pedagogical techniques they employed, or how they were evaluated.

In contrast to the AI techniques used in ITSes, GenAI-powered CSEd tools use LLMs to support open-ended natural language interaction. Other reviews have taken a closer look at specific issues within the GenAI in CSEd space [2, 11, 12, 16]. Bernstein *et al.* [12] focused specifically on harms, surveying papers on GenAI in higher education computing courses and found that nearly a third discussed negative impacts on knowledge acquisition or application, such as overreliance [34, 104], diminished critical thinking [41], and inequitable effects that advantaged already stronger students [81]. Batista *et al.* [11] instead surveyed GenAI's role across higher education broadly, organizing findings around GenAI applications, stakeholder acceptance and perceptions, and specific use situations. The harms identified motivate the development of purpose-built GenAI-powered CSEd tools designed to mitigate them.

2.2 Pedagogical Features and Guardrails for GenAI

GenAI-powered CSEd tools frequently implement what Liu *et al.* [62] describe as "pedagogical guardrails:" design features that guide students toward solutions instead of providing them directly. One such feature is *retrieval-augmented generation* (RAG) [58], which grounds a tool's responses in defined materials so that replies are consistent with course content. To prevent misuse, tools also apply *prompt injection protections* that reject attempts to override the tool's instructions and *usage throttling* that limits how frequently a student can query the tool [62, 105]. A fourth feature, *answer withholding*, appears in a tool from Birillo *et al.* [13], which delivers scaffolded next-step hints rather than disclosing full solutions to programming tasks. Some tools structure the interaction itself as a guardrail through *Socratic questioning*, prompting student reasoning with generated questions rather than delivering answers [40, 95], and through *pacing controls*, which Tran *et al.* [95] used to regulate how much guidance a student received per interaction, finding that more constrained pacing shifted students toward more deliberate problem-solving. Iris [9] combines several of these features with *context-aware feedback*, reading a student's code, the problem statement, and build reports to tailor its responses more closely to the student's situation than a commercial tool would.

Although these features are well documented in individual tools, the rapid pace of tool development means no systematic account yet exists of how pedagogical features and guardrails vary across the broader landscape of GenAI-powered CSEd tools, and no prior review has examined how these tools are built, what populations they serve, or how their creators evaluate them.

Students have expressed a preference for tools that scaffold rather than solve [22], but overly restrictive guardrails risk pushing students toward commercial tools that do not impose such constraints. Kapoor *et al.* [43] designed an "AI TA" that let students bypass scaffolded interactions and view code solutions immediately, and found that 50% of participants used the bypass feature at least once, with low-performing students opting for immediate solutions most frequently. Denny *et al.* [22] similarly found that students wanted the freedom to request direct help when they felt stuck, valuing tools that let them choose their level of support over tools that enforced a single interaction style.

3 Methodology

We conducted an SLR of GenAI-powered tools in postsecondary CS education, informed by the best practices developed by Kitchenham & Charters [49]. Our strategy for screening and determining paper eligibility follows the PRISMA 2020 guidelines [75]. Supplementary

materials, including exact search strings used, codebook, and coding of included papers, are available in our online repository[1].

3.1 Search Scope

3.1.1 Information sources. We examined the most widely used digital libraries for research in CSEd, including ACM Digital Library [1], IEEE Xplore Digital Library [37], Scopus [26], and Springer Link [89].

3.1.2 Inclusion and exclusion criteria. Table 1 summarizes our criteria. We focused on papers that report the development and analysis of an original, GenAI-powered tool for helping postsecondary students with CS. We excluded papers describing tools for non-CS, general course contexts, as well as papers about tools for groups other than postsecondary students, such as K-12 students. Further, the included GenAI-powered CSEd tools must be intended primarily for student use, and should be used to assist with coursework and reinforce course concepts. We did not consider systems designed primarily for evaluating code correctness or assessing student work.

3.1.3 Publication date constraints. We conducted our digital library search on May 5, 2026, and constrained our search to papers published between 2023 and 2026 (inclusive) to focus on tools that implemented modern LLMs (such as OpenAI's GPT-4 [74]). While there is a wide breadth of research on AI-assisted tutors and ITSes that use AI methods beyond just GenAI, the purpose of this SLR is to capture how instructors and researchers have developed tools in response to the rise of commercial tools like ChatGPT, which was not released to the public until November 2022 [73].

Table 1: Inclusion and exclusion criteria.

Inclusion Criteria	Exclusion Criteria
IC_1: Paper is peer-reviewed in an academic conference or journal.	**EC_1:** Paper is not accessible online, or full text is not available.
IC_2: Paper is written in English.	**EC_2:** Paper is too short (<4 pages double-column or <8 pages single-column).
IC_3: Paper reports the development of an original GenAI-powered CSEd tool (not an opinion paper or literature review)	**EC_3:** Tool does not implement a modern LLM, such as GPT-4.
IC_4: Tool is designed for CS education at the postsecondary level.	**EC_4:** Tool is not designed to be used directly by students (e.g., instructor-managed tool)
IC_5: Tool is designed to aid student learning through pedagogical features.	**EC_5:** Tool is designed for autograding or assessing student work.

3.2 Screening Process

3.2.1 Search strings. Our search string was a concatenation of five categories of search terms covering the scope of GenAI-powered CSEd tools, along with our publication date constraint. We used five well-known papers in the GenAI-powered CSEd tool space [9, 46, 61–63] as seed papers for gathering relevant search terms.

- **AI**: "ai" OR "artificial intelligence" OR "llm" OR "large language model"
- **Tooling**: "tutor" OR "coding tool" OR "coding assistant" OR "virtual tutor" OR "teaching assistant" OR "programming assistant" OR "chatbot"
- **Pedagogy**: "pedagogical" OR "educational" OR "learning" OR "learner-focused" OR "learner-centric" OR "guardrail"
- **CS Education**: "computer science education" OR "computing education" OR "programming education"
- **Publication Date**: January 2023 to May 2026 (inclusive)

Search String = (**AI** AND **Tooling** AND **Pedagogy** AND **CS Education** AND **Publication Date**)

Due to a misunderstanding of the search interface, we searched the libraries inconsistently. ACM DL and Springer Link queried the full-text paper, IEEE Xplore used paper metadata, and Scopus used paper titles, abstracts, and keywords.

3.2.2 Search results. Our search yielded a total of 1,834 results. The first author confirmed that each of our five seed papers was included in these results. The first author applied our criteria to the title and abstract of each paper, resulting in 152 remaining papers, 35 of which were removed as duplicates, for a total of 117 papers. The 117 papers were divided among four authors to read the full texts, determine eligibility using our inclusion and exclusion criteria, and code them if the criteria were met. During this process, 45 papers were removed for not meeting our criteria, resulting in a final total of 72 papers. Figure 1 summarizes our process of identifying and screening papers.

3.3 Data Analysis

3.3.1 Codebook calibration. We used inductive coding to create our codebook, starting with an initial list of categories. Three authors read an initial 20% sample of the 117 included papers after duplicate removal (N=24) to create codes and refine definitions; this larger sample was taken to account for the wide variation of tools in the literature and avoid over-sampling a single tool archetype or context. Following this, four authors read a new 10% sample of included papers (N=12) to calculate inter-rater reliability (IRR) using Krippendorff's α [51], resulting in an overall value of 0.704. This value, indicating only moderate agreement, prompted an additional round of calibration and revisions to the codebook by the four authors, who read a second 10% sample of included papers (N=12). Our Krippendorff's α from this sample was 0.813, indicating high agreement. Despite improved overall reliability, agreement varied substantially across individual codes. Following the recommended threshold for reliable data [50], we excluded codes with α less than 0.667 and only report results from codes above that threshold. After excluding these codes, our overall α was 0.861. A breakdown of α values for each category for the retained codes is shown in Table 2.

3.3.2 Codebook categories. **Problem space** refers to the factors motivating the paper authors to develop a GenAI-powered CSEd tool. These could include issues identified with commercial tools,

[1] https://du.ke/genai-powered-postsecondary-csed-tools-supplementary

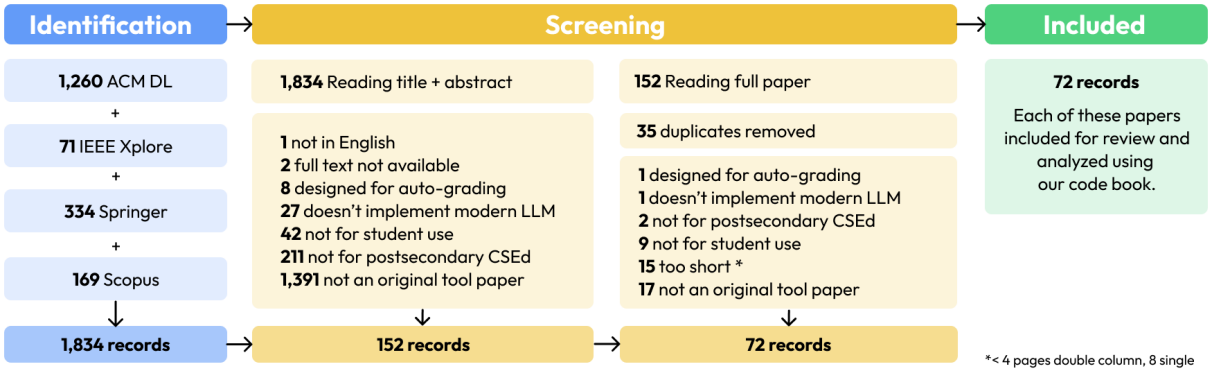


Figure 1: PRISMA diagram specifying identification, screening and inclusion of papers from digital libraries.

Table 2: Krippendorff's α values by code category.

Code category	Overall α	Range
Inclusion/exclusion	0.826	0.826-0.826
Problem space	0.840	0.765-1.000
Educational framings	0.844	0.679-1.000
Pedagogical features	0.854	0.700-1.000
Delivery method	0.823	0.700-1.000
Evaluation goals	0.851	0.707-1.000
Evaluation methods	0.842	0.700-1.000

such as LLM hallucinations, data privacy risks, and concerns of student overreliance, as well as broader classroom challenges, such as high course enrollments limiting students' opportunities for individualized help-seeking.

Educational framings are the theories and concepts from learning science, behavioral science, and education that contextualize the approach taken to develop a tool. Educational frameworks with specific origins are included, such as Bloom's Taxonomy [14] or Vygotsky's Zone of Proximal Development (ZPD) [97], as well as broader topics like metacognition [64], scaffolding [100], self-regulated learning [64], and cognitive load theory [91].

Pedagogical features consist of the technical aspects of the tool that differentiate it from a commercial one and help students learn CS. This category spans the design decisions made to support learning, such as providing scaffolded hints or feedback to students, as well as guardrails to regulate behavior, such as filtering out inappropriate requests or rate-limiting students' queries.

Delivery method is how a student uses the tool, such as through a standalone web application, a feature embedded in an existing course website or learning platform, or an IDE plugin.

Evaluation goals connect to the underlying research goals that are addressed by a paper. Some evaluations are focused on characterizing students' perceptions of a tool or understanding how often students use it, while others seek to measure the extent to which a tool impacts student learning.

Evaluation methods capture how the tool is empirically evaluated. Some papers emphasize measuring learning gains, such as through pre- and post-tests of programming concepts, while others administer surveys to student participants or analyze usage data collected by the tool.

For our results, papers could have multiple codes from a given category; for example, a paper could mention several problem spaces or reference several educational framings. Similarly, papers could have none of the codes present in a given category. Delivery method was the only category where codes were mutually exclusive.

4 Results

4.1 RQ1: Motivations and approaches

4.1.1 Motivations. We identified 6 problem spaces discussed in the context of GenAI-powered CSEd tools (see Table 3). The frequency distribution of problem spaces forms three clusters, with two highly prevalent motivations, two moderately common motivations, and two infrequent motivations.

Table 3: Frequency of problem spaces.

Short name	Problem space	#
Personalized	Desire for personalized, context-aware support	43
Over-rely	Concern that LLMs lead to overreliance	41
Help-seeking	Lack of scalable help-seeking in CS classrooms	27
Immediate	Desire for immediate, always-available support	26
Hallucinate	LLM responses can be hallucinatory	16
Privacy	LLMs have issues with data privacy	3

The desire to provide personalized, context-aware support (Personalized) and concern that LLMs lead to overreliance (Over-rely) were the most commonly identified problem spaces. These problem spaces were mentioned in 60% and 57% of papers, respectively, with 24% of papers (N=17) mentioning both. In one case of Personalized, Arab *et al.* [6] emphasize the difficulty of teaching strategic knowledge, defined as "context-sensitive tactics for how to approach problems". Huo *et al.* [36] also explore Personalized and discuss the benefits that personalized learning can provide to students, arguing the need to "properly use

LLMs to provide personalized subject-matter expertise" and "deliver more precise answers." Regarding Over-rely, several papers detail concerns about using LLMs to write code or solve problems without requiring cognitive effort by the student [7, 67], while others frame the concern around LLMs' abilities to easily complete introductory-level CS homework [30] or students' willingness to accept LLM-generated solutions without verifying them [54, 101].

Lack of scalable help-seeking in CS classrooms (Help-seeking) and desire to provide immediate, always-available support to students (Immediate) were prevalent but less frequent than the preceding problem spaces. Of the 38% of papers mentioning Help-seeking, it was often characterized by the difficulty of a fixed-size teaching staff to provide one-on-one support to students [8, 13], as well as growing enrollment in introductory-sequence CS courses [9, 35, 83]. Immediate was identified in 36% of papers. Jacobs *et al.* [39] and Petula *et al.* [77] stress the importance of feedback being timely to be effective, arguing that LLMs are a promising solution for providing automatic feedback on student work, while Shcherbakov *et al.* [86] affirm that the immediacy of LLM-supported feedback can improve students' motivation and engagement.

The highest co-occurrence of any two problem spaces was between Immediate and Personalized, with 30% of papers (N=22) indicating both. Perez & Ong [76] articulate the connection between these two problem spaces, explaining that GenAI's powerful natural language capabilities allow it to tailor its guidance to a student's own code and questions and do so beyond the time and space constraints of a traditional classroom setting.

Concerns about LLM hallucinations (Hallucinate) were less common than other problem spaces, and worries about data privacy or institutional compliance with LLMs (Privacy) were scarcely mentioned. 22% of papers called attention to Hallucinate, generally describing commercial GenAI tools as poor tutors due to risks of presenting students with inaccurate or misleading information. For example, Rachha & Seyam [82] describe hallucinations as an inherent limitation of GenAI, expressing concerns about "plausible but incorrect or irrelevant content" being treated by students as trustworthy information. Just 4% of papers mentioned concerns about Privacy. Barzanji *et al.* [8] frame the issue specifically in the context of chatbots for CS education, arguing that many "give limited attention to institutional privacy requirements", while Gupta *et al.* [32] and Huo *et al.* [36] express a more explicit concern about students revealing personal information to commercial LLMs.

4.1.2 Approaches. Table 4 summarizes the 8 pedagogical features we identified that GenAI-powered CSEd tools leverage. We divided the pedagogical features into 3 groups: *learning* strategies that support the tutoring and instructional goals of the tool, *constraints* on how a student can use a tool, and *other* features that generally focused on supporting the flow of information between student and tool. We observed *learning* features to be the most common, with fewer uses of *constraint* and *other* features, and most tools integrating 2 or 3 total features.

Learning features were most common across tools, with the most popular pedagogical feature being No-Ans. Providing feedback to students without directly giving away answers was discussed as a pedagogical feature in 81% of papers, with approaches like "CodeHelp" by Liffiton *et al.* [61] adopting a proactive approach to ensure that students were not given code examples that could short-circuit the learning process. In addition to a system prompt instructing the underlying `gpt-3.5-turbo` model not to produce code snippets, CodeHelp leveraged a follow-up prompt for a separate `text-davinci-003` model to double-check the main response for code snippets and remove them.

Table 4: Frequency of pedagogical features by category.

Group	Short Name	Feature	#
Learning	No-Ans	Scaffolded, no-answer feedback	58
	Curate	Curated practice problems	43
	Adapt	Adaptive feedback	41
Constraint	Filter	Filter out inappropriate requests	10
	Over-use	Respond to over-use	8
Other	RAG-Data	Retrieve info from RAG/database	18
	Feed	Students submit feedback	8
	Multi	Multi-modal interaction	4

The next most popular pedagogical feature was the use of curated practice problems (Curate), identified in 60% of papers. These allowed students to engage with instructor-developed learning content or personalized content based on student input. The "Next-Step Hint System" developed by Birillo *et al.* [13], for example, presented students with preset programming assignments broken into smaller tasks, with completion checks and optional LLM-generated hints for each task. Conversely, the "ExPeerience" tool by Zhou *et al.* [106] is an example of student ownership of practice problems: students engage in scaffolded interactions with an LLM to define a problem context and problem description for a worked example in database programming, eventually writing a solution in code that is evaluated by the LLM for correctness.

Closely following Curate, Adapt was identified in 57% of papers and refers to adjustments made by the tool based on prior interactions with the student, similar to how ITSes manage learner state. Cheng *et al.* [17], for example, developed a chatbot that would analyze its conversation history with a student to determine progress made on a given problem and offer "additional hints aligned with the student's specific misconceptions."

Constraint features were uncommon on their own, but were commonly mentioned alongside No-Ans or Adapt. Response filtering (Filter) was identified in 14% of papers, and was implemented to mitigate against off-topic or inappropriate requests from students. For example, as part of the "Iris" tool by Bassner *et al.* [9], an LLM was prompted to evaluate the relevance of student queries on a ten-point scale, and students would receive a response instructing them to rephrase their query if it was scored below a five. Restrictions against over-use (Over-use) were found in 11% of papers; one example from Zhang *et al.* [105] involved a "Silent Mode" on their "TA Agent" that would withhold feedback for a period of time to discourage dependence on the system. These were almost never standalone pedagogical features, with just one paper including Filter with no other features and no papers

only including Over-use. However, the two most frequently co-occurred with No-Ans and Adapt; of the 17 papers that mentioned either Filter or Over-use, 15 also mentioned No-Ans and 11 also mentioned Adapt.

Other features beyond the Learning and Constraint groups were sparse. The use of RAG or a database for information retrieval (RAG-Data) was the most popular feature in this group, identified in 25% of papers. Liu *et al.* [62] implement RAG into their "CS50.ai" suite of tools, using lecture captions from their CS50 course as "a ground-truth external data source" that their tools can reference in responses to students. 11% of papers included a mechanism for students to submit feedback about the tool (Feed). In some cases, the feedback specifically assessed the quality of responses or hints generated by the tool [84], while in others, the feedback was designed to be open-ended and consumed by instructors to inform future tool improvements [103]. Just 6% of tools supported multi-modal interactions with students (Multi); Bamfo *et al.* [7], for example, developed their "interviewBot" tool for conducting mock data structures and algorithms interviews with students, supporting both audio and image input.

Most tools had either 2 or 3 total pedagogical feature codes, with one overwhelmingly popular configuration for each case. 38% of papers (N=27) mentioned 3 pedagogical features that we coded for, while 33% of papers (N=24) mentioned 2. The most common code triplet was {No-Ans, Adapt, RAG-Data}, occurring in 10% of papers (N=7). The most common code pair was {No-Ans, Adapt}, occurring in 25% of papers (N=18); this was also the most common combination of any number of features.

4.1.3 Delivery method. Tools were delivered to students in 5 observed ways, as shown in Table 5. This is the only category with mutually exclusive codes.

Table 5: Delivery method frequency. Seven papers either did not specify a delivery method or used a method outside those listed.

Short name	Delivery method	#
Web	Standalone website/web app	45
Embed	Embedded in course content	8
IDE	IDE plugin/extension	7
Desktop	Standalone desktop app	3
CLI	Command-line tool	2

Deploying the tool as a standalone website or web app (Web) was overwhelmingly the most popular delivery method, appearing in 63% of papers. The second most popular delivery method was substantially less frequent, mentioned in just 11% of papers, and involved embedding a tool into existing course content (Embed). This was achieved by adding the tool to a course discussion forum [36] or an assessment platform [85]. Tools designed as plugins or extensions to an IDE (IDE) were similarly uncommon, appearing in 10% of papers. Finally, standalone desktop applications (Desktop) and command-line tools (CLI) were not popular delivery methods, being found in 4% and 3% of papers, respectively.

4.2 RQ2: Educational framings

Table 6 presents the frequency of the 13 codes we used to classify the educational framings that informed the design of the GenAI-powered CSEd tools. During our coding process, we counted any mention of an educational framing in a paper, even if it was not accompanied by a citation. We observed high co-occurrence between the four most common educational framings, similar usage of informal and formal framings, and a wide range of the number of framings that papers mentioned.

Table 6: Frequency of educational framings.

Short name	Educational framing	#
Scaff	Scaffolding	45
CLT	Cognitive load theory	26
Meta	Metacognition	26
SRL	Self-regulated learning	23
ZPD	Zone of Proximal Development	18
Construct	Constructivist learning theory	16
Self-E	Self-efficacy	15
Socratic	Socratic method	13
Comp-think	Computational thinking	12
Bloom	Bloom's Taxonomy	8
Peer	Peer learning	7
Active	Active learning	6
Collab	Collaborative learning	5

Common educational framings that co-occurred frequently were Scaff, CLT, Meta, and SRL. Scaff and Meta had the highest co-occurrence of any pair of educational framings, with 31% of papers (N=22) mentioning both. This was followed closely by Scaff and SRL (26%, N=19) and Scaff and CLT (25%, N=18).

Scaff was the most popular educational framing and appeared in a range of contexts. For example, Cheng *et al.* [17] describe their tool as a "scaffolding-based, LLM-driven chatbot", connecting the work of Van de Pol *et al.* [96] to the tool's use of hints and feedback on incorrect answers. Conversely, Xie *et al.* [102] do not cite a specific interpretation of scaffolding but instead describe it as the use of constraints that ensures students only receive "minimum necessary support" to preserve independent problem-solving.

We did not identify any patterns distinguishing the use of formal educational theories from informal framings. There was no clear trend mentioning a greater number of formal theories (such as ZPD or Bloom) than informal or very broad framings (such as Peer or Meta) or vice versa. Further, there were no prevailing groupings of framings across the papers. We observed 48 unique combinations of two or more framings.

Educational framings focused on collaboration were the least frequent, reflecting a context of digital learning environments. Two of the least popular educational framings we observed were Peer (10%), and Collab (7%), which have overlapping definitions. This finding is not particularly surprising given that most commercial GenAI tools are constrained to single-user interaction patterns. In one example of Collab, Wang *et al.* [98] investigate the capacity of GenAI to support collaborative learning in groups. They developed a process in which students work

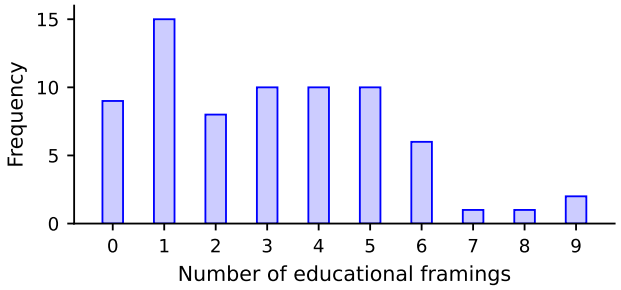


Figure 2: Frequency of number of educational framings mentioned per paper.

together to complete tasks like "task analysis, goal setting, and role allocation" without GenAI, then bring an "AI-Agent" in the loop to provide support with programming tasks and facilitate further student collaboration. Further, Dobson & Karkalas [24] argue that tools should behave like peers that "prompt reflection" and "point to conceptual boundaries" rather than systems designed for revealing answers, applying these principles to their "prototype GenAI tutor".

The prevalence of educational framings varies greatly, with a range of zero to nine framings across papers and a median of three framings mentioned. Figure 2 shows the frequencies of the number of educational framings mentioned in the papers; 14% of papers (N=10) followed the median and mentioned 3 framings. It was most common for papers to reference just 1 educational framing, which occurred in 21% of papers (N=15).

13% of papers (N=9) did not reference any educational framings that we coded for. The absence of these framings reflects the diversity of contexts that inform the development of GenAI-powered CSEd tools. One such example from Roest *et al.* [84] examined the role that "next-step hints" can play in programming education, referencing prior research on automated feedback from Narciss [70] and others [33, 48] to build their "StAP-tutor" tool that offers personalized, LLM-generated hints. In another case, the "Ladder" tool from Kumamoto *et al.* [53] builds on the "design recipe" introduced by Felleisen *et al.* [27], applying the step-by-step process for writing a function to a conversational AI interface that walks students through each step.

4.3 RQ3: Evaluations

4.3.1 Evaluation goals. Table 7 presents the frequencies of the 5 evaluation goals we coded for. Most evaluations sought to understand how students perceived and used tools, with a smaller number of papers seeking to measure tool quality or a tool's impact on learning outcomes.

The most frequent number of evaluation goals for a paper to define was 2, with 46% of papers defining 2 goals (N=33). {Per-S, Use} was the most popular combination of any number of evaluation goals, occurring in 22% of papers (N=16), followed by {Per-S, Learn} and {Per-S, Use, Qual}, both of which occurred in 11% of papers (N=8). Perceptions of the tool from non-student groups (Per-N) was the least common evaluation goal, appearing in 18% of papers. There was some variation in the groups considered for Per-N, including industry professionals [69], university faculty [82], and teaching assistants [71].

Table 7: Frequency of evaluation goals.

Short Name	Goal	#
Per-S	Understand student perceptions	57
Use	Understand student usage patterns	44
Qual	Analyze quality of tool output	28
Learn	Measure impact on learning outcomes	24
Per-N	Understand non-student perceptions	13

The goals Qual and Learn were more ambiguous than the three others, with varying definitions of how to achieve them. In one Qual example, Hou *et al.* [35] defined the quality of the output of their "CodeTailor" tool based on the correctness, personalization, and conciseness of Parsons puzzles generated from students' incorrect code. Similarly, Taylor *et al.* [94] analyzed explanations generated by their "dcc –help" tool according to seven criteria: conceptual accuracy, presence of inaccuracies, correctness, relevance, completeness, presence of code, and response type.

In the context of Learn, learning outcomes were interpreted both as quantifiable learning gains as well as indications of learners' affective states. Stienstra *et al.* [90], for example, sought to understand how students who completed programming exercises with the help of their "virtual tutor" would perform independently on a quiz of the topics they had just practiced. Similarly, Gu *et al.* [31] defined "learning performance" according to changes in scores on a JavaScript test administered once before and once after their intervention. Sooriamurthi *et al.* [88], on the other hand, wanted to understand how their tool changed students' self-efficacy and nervousness over time, measuring these with surveys taken at three points during a semester.

4.3.2 Evaluation methods. Table 8 shows the frequencies of the 9 evaluation methods we captured, as well as the frequency of papers reporting no evaluation. Post-surveys and logging tool usage data were the two most prevalent methods, and they were frequently used together. Methods with structured experimental designs like post-assessments or completing tasks in a think-aloud were less common but still prevalent, and external testing frequently complemented other student-centered methods.

Post-surveys after the evaluation period (Post-S) were most common, but pre-surveys (Pre-S) were not as prevalent. Some evaluations measured student perceptions of the tool with Likert-scale and free-response questions, such as in an evaluation conducted by Tang *et al.* [92]. Others leveraged existing survey instruments such as the Technology Acceptance Model [21], System Usability Scale [15] or Partner Model Questionnaire [25]. While post-surveys were included in 78% of papers, just 32% of papers included pre-surveys. Pre-surveys often served a different purpose than post-surveys; for example, Perez & Ong [76] deployed a pre-survey to gather information on participants' backgrounds and prior programming experience. Among papers that included a post-survey, 38% of them (N=21) included a pre-survey as well.

Table 8: Frequency of evaluation methods.

Short Name	Method	#
Post-S	Post-survey after study period	56
Log	Logging usage data	53
Pre-S	Pre-survey before study period	23
Res	Complete tasks in restricted setting	23
Ext	External testing	21
Post-A	Post-assessment after study period	19
Unres	Complete tasks in unrestricted setting	13
Pre-A	Pre-assessment before study period	12
Open	Open access to tool	8
None	No evaluation	2

Most evaluations used between 2 and 4 methods, frequently combining post-surveys (Post-S) with analysis of usage data (Log). 69% of papers (N=50) used between 2 and 4 evaluation methods, with 3 methods being the most common arrangement (N=22). The most frequently occurring combination of methods was {Pre-S, Post-S, Res, Log}, which was the case for 6 papers. This was followed closely by {Post-S, Log}, {Post-S, Log, Unres}, and {Post-S, Log, Ext}, each of which occurred in 5 papers. There was not, however, an overwhelmingly popular combination of evaluation methods; there were 42 distinct combinations.

Evaluations frequently included structured requirements for participants, with a wide range of experimental designs. 32% of papers conducted evaluations with students in a restricted experimental setting (Res); Jacobs *et al.* [38], for example, conducted a two-hour procedure with students following a think-aloud protocol and using eye-tracking and screen recording software. Evaluations involving task completion in unrestricted settings (Unres) were identified in 18% of papers and were more flexible than Res, such as giving students assignments that leverage the tool that they complete on their own time [7, 19, 52].

In terms of quantifying the impact of a tool with instruments like concept tests or programming quizzes, 26% of papers (N=19) included pre-assessments (Pre-A) or post-assessments (Post-A). Pre-A and Post-A were frequently coupled together, with 63% of papers that performed Post-A also performing Pre-A, and no papers only performing Pre-A. One use of assessments from Alanzi *et al.* [3] involved Pre-A to assess participants' programming skills and comparing these results to a Post-A after the intervention of their "PyChatAI" tool to "measure changes in coding skills and problem-solving abilities."

In contrast to these structured methods, evaluations involving open access to a tool (Open) were much less common. In one example of Open, Li & Bao [59] deployed their "LMS-integrated tutoring interface" for twelve weeks, using Pre-A and Post-A to measure the tool's impact on learning outcomes. Other Open evaluations were conducted in the context of limited deployments of a tool, such as Nieto-Cardenas *et al.* [71] running a pilot of "Owlgorithm" for a single semester or Zhu *et al.* [107] launching "FEEDBOT" for just part of a semester with a focus on identifying bugs and making tool improvements.

A significant number of papers tested their tools with methods external to CS courses (Ext), like constructed benchmarks or expert evaluations, but also complemented these with student-focused evaluations. Jury *et al.* [42], for example, conducted an "expert assessment" of fourteen LLM-generated worked examples from their "WorkedGen" tool, assessing the examples according to rubric metrics like explanation clarity, code correctness, and code readability. Following this, they also conducted a user study with introductory-level CS students to better understand their perceptions of WorkedGen. Similarly, Ma *et al.* [65] evaluated their "RAGMan" AI tutor by reviewing a sample of student conversations with the tool, qualitatively analyzing how often the tool produced "good" responses to in-scope and out-of-scope queries. Additionally, Ma *et al.* analyzed student survey feedback and compared the grade distributions of class offerings that used the tool with those that did not. Ext is almost never a standalone evaluation method; just 10% (N=2) of papers using constructed benchmarks did not include other methods, while 81% of them (N=17) also performed an evaluation based on logged usage data and 62% of them (N=13) conducted post-surveys after an evaluation period.

5 Discussion and Future Work

The papers we reviewed reflect a field exploring a wide range of architectures, educational framings, and evaluation designs for GenAI-powered postsecondary CSEd tools. Section 4 reports what our coding scheme captured across the paper corpus. This section goes further, describing patterns we observed while reading these papers that our coding scheme did not explicitly capture, as well as detailing opportunities for future research and tool development.

5.1 Motivations and Approaches

5.1.1 LLM choice. The choices of LLMs used in GenAI-powered CSEd tools reflected the pace of commercial model development more than deliberate, pedagogically-driven selections. OpenAI's GPT models were the most prominently used, with a smaller number of tools using other commercial models, open-weight models (models whose weights are publicly released, enabling local hosting without reliance on third-party APIs), or institution-hosted alternatives. Model choice was rarely discussed in pedagogical terms, more often justified by cost or availability; whether model selection shapes tool quality remains an open question. As more capable models become cheaper and open-weight alternatives more accessible, future work should examine whether model choice impacts factors like pedagogical feature design or tool usability.

5.1.2 System design. Prompt engineering was the most common strategy for shaping tool behavior, with many tools relying on detailed system prompts, chain-of-thought prompting [99], or few-shot examples to constrain outputs. RAG grounded responses in course-specific materials, and multi-agent pipelines split tutoring responsibilities across specialized agents in more complex deployments. While these techniques can improve a tool's effectiveness, they also come with tradeoffs. RAG, for example, can help align tool responses to course content and students' expected knowledge, but it also requires instructors to curate and maintain a large body of information. Prompt-only approaches deploy faster, but provide

weaker guarantees that responses will remain course-specific or avoid introducing material beyond the intended scope. Future work should examine how architectural decisions, such as the choice between RAG and prompt-only approaches, affect tool quality.

5.1.3 Course context. Course context showed a clear skew toward introductory programming. The majority of the tools we studied were deployed in introductory-sequence CS courses, with a smaller number deploying in courses designed for a specific programming language like Python or C. Tools designed for advanced undergraduate and graduate courses were exceptions [85, 105]. Future work should investigate the role that GenAI-powered CSEd tools can play in those settings, examining how to support more advanced computing concepts, address misconceptions specific to particular subfields, and complement students' greater baseline knowledge and autonomy.

5.2 Educational Framings

5.2.1 Variation in theoretical grounding. We did not observe a clear connection between the scope of a tool and the number of educational framings used to contextualize the tool's development. Tools built for a narrow task, such as hint generation or error message explanation, operate at a different scope than a general-purpose tool, and their use of educational framings varied widely independent of that scope. Future work building tools would benefit from engaging more deliberately with prior education research. Connecting tool features to established knowledge about how students learn can help researchers select appropriate instructional strategies, anticipate barriers to learning, and design interactions that support learning rather than merely deliver information.

5.2.2 Operationalization of framings. Across the papers we examined, scaffolding appeared more often than any other educational framing. However, papers often mentioned scaffolding and other educational framings without clearly defining how they interpreted the framing or how it informed specific design decisions. This ambiguity makes it difficult to determine what pedagogical mechanism a feature is intended to support, assess whether the implementation reflects the cited framing, or build cumulatively on the findings.

Scaffolding, for example, provides temporary support that helps students perform within their Zone of Proximal Development, but that support can take many forms. Similarly, active learning can be characterized through multiple techniques such as peer instruction, think-pair-share, and minute papers, and self-regulated learning often serves as an umbrella term for other related theories [64]. Future work should include more deliberate use of educational framings, connecting them to specific tool features and designing evaluations that demonstrate how the framing impacted tool efficacy. Doing so would make theoretical grounding more than a label: it would provide a rationale for design decisions and identify outcomes that an evaluation should measure.

5.3 Evaluations

5.3.1 Evaluation instruments and learning outcomes. Post-surveys capturing student satisfaction or tool usefulness were far more common than post-assessments measuring learning outcomes. Learning gains proved difficult to measure cleanly in several cases. Some tools relied entirely on perception measurements, collecting post-deployment surveys without direct assessment of what students actually learned. Others used post-assessment performance as a proxy for learning without a pre-assessment or control condition to confirm the gain came from the tool rather than other factors.

Papers also differed in how explicitly they acknowledged the limitations of their evidence for claims about learning. Some named the gap directly, acknowledging that self-reported perception could not substitute for measured learning, while others made stronger claims from the same kind of evidence. As a community, we need to reach consensus on what standard of evidence we expect for different claims about a tool. As GenAI tool research matures, the novelty of implementing a basic LLM-powered tutor may no longer be sufficient on its own to establish a research contribution. Studies whose primary finding is that students have positive perceptions towards the tool may be better framed as experience reports unless they also provide evidence on learning, behavior, design mechanisms, or another substantive research question.

5.3.2 Control groups and comparison conditions. In evaluations of the tools we reviewed, a comparison condition could help researchers attribute observed differences to the tool rather than to practice, exposure to the material, the passage of time, or other features of the study. Not every paper set out to measure impact relative to an alternative; some studies aimed only to understand how students used and felt about a new tool, without a comparison condition in mind. For studies that do make comparative claims, however, the absence of a control condition limits how much weight those claims can carry.

Future work should consider commercial GenAI tools as a possible control condition. One motivating case is when the practical decision facing instructors concerns which form of AI assistance to permit. A related question that received little attention was why students would choose a purpose-built tool over a commercial one in the first place, which a commercial-tool control group is well-suited to answer, but other experimental designs are not. Researchers should select a comparison condition based on the mechanism their study aims to isolate. When the practical question concerns whether instructors should provide a purpose-built tool rather than permit commercial GenAI access, the commercial tool is a particularly meaningful comparison because it also reveals whether students would choose the purpose-built alternative.

5.4 Recommendations

We offer a set of recommendations for researchers and instructors considering developing their own GenAI-powered postsecondary CSEd tools.

5.4.1 Start with the problem, not the tool. Across the papers we reviewed, the most compelling GenAI-powered CSEd tools were motivated by specific needs observed in CS classrooms and adopted features that directly addressed those needs. An instructor or researcher observing commercial tools completing programming assignments on behalf of students, for example, could develop a tool with guardrails that encourage independent problem-solving. On the other hand, weaker tools were much more solution-oriented, prioritizing the integration of GenAI without fully considering how

or why this should occur. Needs finding principles from research in learner-centered design [72, 87] should be applied to tool development, helping reveal what is necessary for students to meet the learning objectives of their CS coursework and what kind of solution can fit those needs.

5.4.2 Build on what the field already knows. Engaging with prior work on GenAI-powered CSEd tools, as well as the findings that informed their development, will help ensure that new tools are both effective and novel. Developing a worthwhile tool requires understanding what other tools have gotten right and wrong already. Techniques for prompt engineering, filtering out unhelpful responses, minimizing hallucinations, and the like have been described in detail in many papers, and should be viewed as minimum standards for a quality tool. Further, referencing research in CSEd, learning science, cognitive science, and other fields can provide a robust theoretical grounding for a tool's pedagogical features, aligning them with known approaches to support learning. By acknowledging and building on existing knowledge, researchers and instructors can avoid reinventing the wheel and help produce meaningful contributions to the GenAI-powered CSEd tool space.

5.4.3 Investigate the gaps. Though GenAI-powered CSEd tools are still an emerging research area, we are already observing a skew toward tools supporting introductory-sequence CS courses. Many such tools include pedagogical features that place constraints on student-tool interactions, such as limiting free-form text inputs or only providing curated practice problems. While these features may be helpful for students who need greater scaffolding to develop foundational computer science skills, they are not a one-size-fits-all solution for integrating GenAI in CSEd and do not fit the needs of other groups such as advanced undergraduates or graduate students. Researchers and practitioners should expand the scope of GenAI-powered CSEd tools, examining how the tools can promote learning in contexts beyond the introductory level. Future tool development should also support competencies like software engineering, code review, system design, and AI-assisted programming.

5.4.4 Consider that students have alternatives. Few papers confronted what we view as the central tension for GenAI-powered CSEd tools: they compete for student attention against commercial tools that offer sophisticated features while imposing fewer guardrails. Students will always have commercial tools as an alternative in unsupervised settings, limiting the efficacy of tools built by researchers and instructors. Tool builders must be mindful of this constraint and make thoughtful design decisions to encourage student adoption of their tools. Using RAG to ground tool responses in course contexts, embedding tools in platforms students already use, and making regular improvements in response to student feedback are just some of the strategies worth considering. Further, comparative evaluations with commercial tools would illuminate when students prefer instructor-built tools, and what pedagogical features and deployment approaches will nudge students toward learning-oriented options.

6 Limitations

We made several scoping decisions that limit the conclusions we can draw from our findings. First, we restricted our search to papers written in English, which may have excluded relevant works published in other languages. Second, our search was limited to four major digital libraries, which may have excluded relevant scholarship from other communities such as ASEE PEER [4] or pre-prints published to arXiv [18]. Third, we did not perform snowballing on papers from our search results, so we may have missed relevant papers not captured by our search string. Finally, only the first author applied our inclusion and exclusion criteria to the titles and abstracts of papers in our search results, while four authors read the remaining papers in full. Taken together, these factors may introduce some bias and limit generalizability. In addition, we reviewed only papers published between January 2023 and May 2026. Given the rapidly evolving GenAI tool landscape, new tools, architectures, and evaluation practices have almost certainly emerged since our search, and our findings may not reflect the field's current state.

Another limitation is that our search process was not consistent in searching the digital libraries' contents. This discrepancy in how we applied our search string could have inadvertently excluded relevant papers.

Additionally, decisions made during coding limit the precision of some findings. For example, our coding of educational framings did not distinguish between those that included a formal citation and those that did not. As a result, our reporting of framing counts cannot speak to how rigorously or how intentionally a given framing may have informed a tool's design.

Further, there were some tool attributes that we could not reliably operationalize as codes and calculate an IRR for due to the complexity of the data; examples of this are the architectural patterns of tools and the statistical analysis methods used for tool evaluations. We did not report these attributes in Section 4, instead reflecting on limited observations in Section 5.

7 Conclusion

We analyzed 72 papers to understand trends in GenAI-powered post-secondary CSEd tool development, educational framings, and evaluations.Our key findings include that tool creation is frequently motivated by desires to provide immediate and personalized learning support to students, as well as concerns that commercial tools can lead to overreliance. The tools we examined emphasized learning-oriented features designed to avoid giving direct answers and adapt to students' interactions over time.Further, tool development was informed by educational framings such as scaffolding, cognitive load theory, active learning, and self-regulated learning.Tools were evaluated with a wide range of goals, such as to understand student perceptions and usage patterns or to measure learning gains, and these evaluations were conducted using methods like surveys, assessments, lab studies, and expert reviews.We discuss potential future directions for GenAI-powered CSEd tool research, including tool use in advanced CS coursework and comparative analysis with commercial tools. Finally, we advocate for tool development practices that emphasize thorough exploration of student needs, rich connections to existing research, and robust evaluations.

References

[1] ACM. 2026. ACM Digital Library. https://dl.acm.org/. Accessed: 2026-03-24.

[2] Friday Joseph Agbo, Chris Olivia, Godsalvation Oguibe, Ismaila Temitayo Sanusi, and Godwin Sani. 2025. Computing education using generative artificial intelligence tools: A systematic literature review. *Computers and Education Open* 9 (Dec. 2025), 100266. doi:10.1016/j.caeo.2025.100266

[3] Manal Alanazi, Ben Soh, Halima Samra, and Alice Li. 2025. PyChatAI: Enhancing python programming skills—an empirical study of a smart learning system. *Computers* 14, 5 (2025), 158. doi:10.3390/computers14050158

[4] American Society for Engineering Education. 2026. Papers on Engineering Education Repository (PEER). https://peer.asee.org/. Accessed: 2026-07-13.

[5] Anthropic. 2026. The AI for Problem Solvers | Claude by Anthropic. https://claude.com/product/overview

[6] Maryam Arab, Hanning Li, Rushal Butala, and Steve Oney. 2025. Co-Advisor: Learning Programming Strategies in Context . In *2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 343–354. doi:10.1109/VL-HCC65237.2025.00045

[7] Kwabena Bamfo, Olaf Hall-Holt, Oluwakemi Ola, Dennis Owusu, and Govindha Yeluripati. 2026. Process-Oriented Homework with an Instructor-Designed AI-Bot: An Early Experience in a Data Structures and Algorithms Course. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1* (USA) *(SIGCSE TS 2026)*. Association for Computing Machinery, New York, NY, USA, 87–93. doi:10.1145/3770762.3772659

[8] Chrakhan Barzanji, Niclas Rosteck, Sebastian Eberl, Sebastian Rottmann, and Claudia Loitsch. 2026. ST-Buddy: Designing and Evaluating a Course-Grounded LLM Chatbot for Academic and Administrative Support. In *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*. Association for Computing Machinery, New York, NY, USA, Article 574, 6 pages. doi:10.1145/3772363.3798558

[9] Patrick Bassner, Eduard Frankford, and Stephan Krusche. 2024. Iris: An AI-Driven Virtual Tutor for Computer Science Education. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*. Association for Computing Machinery, New York, NY, USA, 394–400. doi:10.1145/3649217.3653543

[10] Patrick Bassner, Ben Lenk-Ostendorf, Ramona Beinstingel, Tobias Wasner, and Stephan Krusche. 2026. Less stress, better scores, same learning: The dissociation of performance and learning in AI-supported programming education. *Computers and Education: Artificial Intelligence* 10 (June 2026), 100537. doi:10.1016/j.caeai.2025.100537

[11] João Batista, Anabela Mesquita, and Gonçalo Carnaz. 2024. Generative AI and higher education: Trends, challenges, and future directions from a systematic literature review. *Information* 15, 11 (2024), 676. doi:10.3390/info15110676

[12] Seth Bernstein, Ashfin Rahman, Nadia Sharifi, Ariunjargal Terbish, and Stephen MacNeil. 2025. Beyond the Benefits: A Systematic Review of the Harms and Consequences of Generative AI in Computing Education. In *Proceedings of the 25th Koli Calling International Conference on Computing Education Research (Koli Calling '25)*. Association for Computing Machinery, New York, NY, USA, 1–18. doi:10.1145/3769994.3770036

[13] Anastasiia Birillo, Elizaveta Artser, Anna Potriasaeva, Ilya Vlasov, Katsiaryna Dzialets, Yaroslav Golubev, Igor Gerasimov, Hieke Keuning, and Timofey Bryksin. 2024. One Step at a Time: Combining LLMs and Static Analysis to Generate Next-Step Hints for Programming Tasks. In *Proceedings of the 24th Koli Calling International Conference on Computing Education Research (Koli Calling '24)*. Association for Computing Machinery, New York, NY, USA, Article 9, 12 pages. doi:10.1145/3699538.3699556

[14] Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. 1956. *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain.* David McKay Company, New York.

[15] John Brooke et al. 1996. SUS-A quick and dirty usability scale. *Usability evaluation in industry* 189, 194 (1996), 4–7.

[16] Chi In Chang, Wan Chong Choi, and Iek Chong Choi. 2025. A Systematic Literature Review of the Opportunities and Advantages for AIGC (OpenAI ChatGPT, Copilot, Codex) in Programming Course. In *Proceedings of the 2024 7th International Conference on Big Data and Education (ICBDE '24)*. Association for Computing Machinery, New York, NY, USA, 29–35. doi:10.1145/3704289.3704301

[17] Gary Cheng, Winnie Wong, Lumen Luo, and Marvin Yu. 2025. Integrating a Scaffolding-Based, LLM-Driven Chatbot into Programming Education: A University Case Study. In *2025 International Symposium on Educational Technology (ISET)*. 1–5. doi:10.1109/ISET65607.2025.00047

[18] Cornell University. 2026. arXiv e-Print Archive. https://arxiv.org/. Accessed: 2026-07-13.

[19] Aaron S Crandall, Gina Sprint, and Bryan Fischer. 2023. Generative pre-trained transformer (GPT) models as a code review feedback tool in computer science programs. *Journal of Computing Sciences in Colleges* 39, 1 (2023), 38–47.

[20] Tyne Crow, Andrew Luxton-Reilly, and Burkhard Wuensche. 2018. Intelligent tutoring systems for programming education: a systematic review. In *Proceedings of the 20th Australasian Computing Education Conference* (Brisbane, Queensland, Australia) *(ACE '18)*. Association for Computing Machinery, New York, NY, USA, 53–62. doi:10.1145/3160489.3160492

[21] Fred D Davis et al. 1989. Technology acceptance model: TAM. *Al-Suqri, MN, Al-Aufi, AS: Information seeking behavior and technology adoption* 205, 219 (1989), 5.

[22] Paul Denny, Stephen MacNeil, Jaromir Savelka, Leo Porter, and Andrew Luxton-Reilly. 2024. Desirable Characteristics for AI Teaching Assistants in Programming Education. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 408–414. doi:10.1145/3649217.3653574

[23] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (Jan. 2024), 56–67. doi:10.1145/3624720

[24] Lee-Roy Tye Dobson and Sokratis Karkalas. 2026. Exploring Minimally Intrusive GenAI Scaffolding for Introductory Programming Education. In *Proceedings of the 2026 14th International Conference on Software and Information Engineering (ICSIE '26)*. Association for Computing Machinery, New York, NY, USA, 26–34. doi:10.1145/3789595.3789611

[25] Philip R. Doyle, Iona Gessinger, Justin Edwards, Leigh Clark, Odile Dumbleton, Diego Garaialde, Daniel Rough, Anna Bleakley, Holly P. Branigan, and Benjamin R. Cowan. 2025. The Partner Modelling Questionnaire: A Validated Self-Report Measure of Perceptions toward Machines as Dialogue Partners. *ACM Trans. Comput.-Hum. Interact.* 32, 4, Article 39 (Aug. 2025), 33 pages. doi:10.1145/3729170

[26] Elsevier. 2026. Scopus. https://www.scopus.com/pages/home. Accessed: 2026-05-20.

[27] Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, and Shriram Krishnamurthi. 2018. *How to design programs: an introduction to programming and computing.* MIT Press, Cambridge, MA, USA.

[28] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference (ACE '22)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3511861.3511863

[29] Rodrigo Elias Francisco and Flávio de Oliveira Silva. 2022. Intelligent Tutoring System for Computer Science Education and the Use of Artificial Intelligence: A Literature Review. In *Proceedings of the 14th International Conference on Computer Supported Education (CSEDU 2022)*, Vol. 1. Springer Cham, Cham, Switzerland, 338–345. doi:10.5220/0011084400003182

[30] Kevin Gold and Shuang Geng. 2025. On the Helpfulness of a Zero-Shot Socratic Tutor. In *Artificial Intelligence in Education Technologies: New Development and Innovative Practices*, Tim Schlippe, Eric C. K. Cheng, and Tianchong Wang (Eds.). Springer Nature, Singapore, 154–170. doi:10.1007/978-981-97-9255-9_11

[31] Jian-Hua Gu, Sheng-Chih Chen, Lin Hui, Tipajin Thaipisutikul, Hui-Shiuan Hung, Heng Du, and Chih-Chi Yeh. 2026. Evaluating Design Strategies for AI Virtual Teaching Assistants and Immersive Learning Scaffolding in Programmed Teaching Games. In *Ubi-Media Computing, Pervasive Systems, Algorithms and Networks*, Lin Hui, Ching-Hsien Hsu, and Somchoke Ruengittinun (Eds.). Springer Nature Singapore, Singapore, 283–295. doi:10.1007/978-981-96-6294-4_21

[32] Rajan Das Gupta, Md. Tanzib Hosain, M. F. Mridha, and Salah Uddin Ahmed. 2025. Multimodal Programming in Computer Science with Interactive Assistance Powered by Large Language Model. In *Human-Computer Interaction*, Masaaki Kurosu and Ayako Hashizume (Eds.). Springer Nature Switzerland, Cham, 59–69. doi:10.1007/978-3-031-93864-1_5

[33] Qiang Hao, David H. Smith IV, Lu Ding, Amy Ko, Camille Ottaway, Jack Wilson, Kai H. Arakawa, Alistair Turcan, Timothy Poehlman, and Tyler Greer. 2022. Towards understanding the effective design of automated formative feedback for programming assignments. *Computer Science Education* 32, 1 (2022), 105–127. doi:10.1080/08993408.2020.1860408

[34] Irene Hou, Owen Man, Kate Hamilton, Srishty Muthusekaran, Jeffin Johnykutty, Leili Zadeh, and Stephen MacNeil. 2025. 'All Roads Lead to ChatGPT': How Generative AI is Eroding Social Interactions and Student Learning Communities. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2025)*. Association for Computing Machinery, New York, NY, USA, 79–85. doi:10.1145/3724363.3729024

[35] Xinying Hou, Zihan Wu, Xu Wang, and Barbara J. Ericson. 2024. CodeTailor: LLM-Powered Personalized Parsons Puzzles for Engaging Support While Learning Programming. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale* (Atlanta, GA, USA) *(L@S '24)*. Association for Computing Machinery, New York, NY, USA, 51–62. doi:10.1145/3657604.3662032

[36] Huan Huo, Xiaoyu Ding, Zhihao Guo, Sheng Shen, Dayong Ye, Olivia Pham, David Milne, Luke Mathieson, and Anne Gardner. 2024. Accelerating Learning with AI: Improving Students' Capability to Receive and Build Automated Feedback for Programming Courses. In *2024 World Engineering Education Forum-Global Engineering Deans Council (WEEF-GEDC)*. IEEE, 1–9. doi:10.1109/WEEF-GEDC63419.2024.10854928

[37] IEEE. 2026. IEEE Xplore Digital Library. https://ieeexplore.ieee.org/. Accessed: 2026-03-24.

[38] Sven Jacobs, Jan Haas, and Natalie Kiesler. 2025. Student Engagement with GenAI's Tutoring Feedback: A Mixed Methods Study. In *Proceedings of the 25th Koli Calling International Conference on Computing Education Research (Koli Calling '25)*. Association for Computing Machinery, New York, NY, USA, Article 33, 12 pages. doi:10.1145/3769994.3770034

[39] Sven Jacobs, Maurice Kempf, and Natalie Kiesler. 2025. That's Not the Feedback I Need! - Student Engagement with GenAI Feedback in the Tutor Kai. In *Proceedings of the 2025 Conference on UK and Ireland Computing Education Research (UKICER '25)*. Association for Computing Machinery, New York, NY, USA, Article 3, 7 pages. doi:10.1145/3754508.3754512

[40] Hyoungwook Jin, Seonghee Lee, Hyungyu Shin, and Juho Kim. 2024. Teach AI How to Code: Using Large Language Models as Teachable Agents for Programming Education. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 652, 28 pages. doi:10.1145/3613904.3642349

[41] Gregor Jošt, Viktor Taneski, and Sašo Karakatič. 2024. The Impact of Large Language Models on Programming Education and Student Learning Outcomes. *Applied Sciences* 14, 10 (Jan. 2024), 4115. doi:10.3390/app14104115

[42] Breanna Jury, Angela Lorusso, Juho Leinonen, Paul Denny, and Andrew Luxton-Reilly. 2024. Evaluating LLM-generated Worked Examples in an Introductory Programming Course. In *Proceedings of the 26th Australasian Computing Education Conference* (Sydney, NSW, Australia) *(ACE '24)*. Association for Computing Machinery, New York, NY, USA, 77–86. doi:10.1145/3636243.3636252

[43] Amanpreet Kapoor, Paul Denny, Leo Porter, Stephen MacNeil, and Marc Diaz. 2026. Exploring Student Behaviors and Motivations when using AI Teaching Assistants with Optional Guardrails. In *Proceedings of the 28th Australasian Computing Education Conference (ACE '26)*. Association for Computing Machinery, New York, NY, USA, 22–31. doi:10.1145/3786228.3786233

[44] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 455, 23 pages. doi:10.1145/3544548.3580919

[45] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2024. How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3631802.3631806

[46] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 650, 20 pages. doi:10.1145/3613904.3642773

[47] Tyson Kendon, Leanne Wu, and John Aycock. 2023. AI-Generated Code Not Considered Harmful. In *Proceedings of the 25th Western Canadian Conference on Computing Education (WCCCE '23)*. Association for Computing Machinery, New York, NY, USA, 1–7. doi:10.1145/3593342.3593349

[48] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Trans. Comput. Educ.* 19, 1, Article 3 (Sept. 2018), 43 pages. doi:10.1145/3231711

[49] Barbara A. Kitchenham and Stuart Charters. 2007. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Technical Report EBSE-2007-01. School of Computer Science and Mathematics, Keele University.

[50] Klaus Krippendorff. 2004. Reliability in Content Analysis: Some Common Misconceptions and Recommendations. *Human Communication Research* 30, 3 (2004), 411–433. https://doi.org/10.1111/j.1468-2958.2004.tb00738.x

[51] Klaus Krippendorff. 2019. *Content Analysis: An Introduction to Its Methodology* (4 ed.). SAGE Publications, Thousand Oaks, CA.

[52] Shriram Krishnamurthi, Thore Thießen, and Jan Vahrenhold. 2025. Porpoise: An LLM-Based Sandbox for Novices to Practice Writing Purpose Statements. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E* (Singapore, Singapore) *(SPLASH-E '25)*. Association for Computing Machinery, New York, NY, USA, 75–89. doi:10.1145/3758317.3759683

[53] Kouta Kumamoto, Youyou Cong, and Hidehiko Masuhara. 2025. An Interactive Learning Environment for Program Design. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E* (Singapore, Singapore) *(SPLASH-E '25)*. Association for Computing Machinery, New York, NY, USA, 66–74. doi:10.1145/3758317.3759682

[54] Oka Kurniawan, Erick Chandra, Christopher M. Poskitt, Yannic Noller, Kenny Tsu Wei Choo, and Cyrille Jegourel. 2025. Designing for Novice Debuggers: A Pilot Study on an AI-Assisted Debugging Tool. In *Proceedings of the 25th Koli Calling International Conference on Computing Education Research (Koli Calling '25)*. Association for Computing Machinery, New York, NY, USA, Article 41, 7 pages. doi:10.1145/3769994.3769997

[55] Sam Lau and Philip Guo. 2023. From "Ban It Till We Understand It" to "Resistance is Futile": How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1 (ICER '23, Vol. 1)*. Association for Computing Machinery, New York, NY, USA, 106–121. doi:10.1145/3568813.3600138

[56] Nguyen-Thinh Le, Sven Strickroth, Sebastian Gross, and Niels Pinkwart. 2013. A Review of AI-Supported Tutoring Approaches for Learning Programming. In *Advanced Computational Methods for Knowledge Engineering*, Ngoc Thanh Nguyen, Tien van Do, and Hoai An le Thi (Eds.). Springer International Publishing, Heidelberg, 267–279. doi:10.1007/978-3-319-00293-4_20

[57] Angélique Létourneau, Marion Deslandes Martineau, Patrick Charland, John Alexander Karran, Jared Boasen, and Pierre Majorique Léger. 2025. A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education. *npj Science of Learning* 10, 1 (2025), 29. doi:10.1038/s41539-025-00320-7

[58] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[59] Yang Li and Zilong Bao. 2026. Integrating AI Virtual Tutors in Blended Learning: A Controlled Evaluation of Learning Outcomes and Student Engagement. In *Proceedings of the 2025 2nd International Conference on Artificial Intelligence and Future Education (AIFE '25)*. Association for Computing Machinery, New York, NY, USA, 247–251. doi:10.1145/3785987.3786027

[60] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 52, 13 pages. doi:10.1145/3597503.3608128

[61] Mark Liffiton, Brad E Sheese, Jaromir Savelka, and Paul Denny. 2024. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, Article 8, 11 pages. doi:10.1145/3631802.3631830

[62] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 750–756. doi:10.1145/3626252.3630938

[63] Rongxin Liu, Julianna Zhao, Benjamin Xu, Christopher Perez, Yuliia Zhukovets, and David J. Malan. 2025. Improving AI in CS50: Leveraging Human Feedback for Better Learning. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) *(SIGCSETS 2025)*. Association for Computing Machinery, New York, NY, USA, 715–721. doi:10.1145/3641554.3701945

[64] Dastyni Loksa, Lauren Margulieux, Brett A. Becker, Michelle Craig, Paul Denny, Raymond Pettit, and James Prather. 2022. Metacognition and Self-Regulation in Programming Education: Theories and Exemplars of Use. *ACM Trans. Comput. Educ.* 22, 4, Article 39 (Sept. 2022), 31 pages. doi:10.1145/3487050

[65] Iris Ma, Alberto Krone-Martins, and Cristina Videira Lopes. 2024. Integrating AI Tutors in a Programming Course. In *Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 1* (Virtual Event, NC, USA) *(SIGCSE Virtual 2024)*. Association for Computing Machinery, New York, NY, USA, 130–136. doi:10.1145/3649165.3690094

[66] Stephen MacNeil, Juho Leinonen, Paul Denny, Natalie Kiesler, Arto Hellas, James Prather, Brett A. Becker, Michel Wermelinger, and Karen Reid. 2024. Discussing the Changing Landscape of Generative AI in Computing Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2* (Portland, OR, USA) *(SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 1916. doi:10.1145/3626253.3635369

[67] Julio R. Martinez, Mario Chong, Susana Maya, Maria Alejandra Caicedo, and Ana Luna. 2025. Enhancing Algorithmic Thinking and Emotional Resilience in Programming Education Through AI Powered Virtual Tutoring . In *2025 IEEE Frontiers in Education Conference (FIE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–8. doi:10.1109/FIE63693.2025.11328240

[68] Cecily Mauran. 2023. *Meet Copilot, Microsoft's AI tool for work and productivity.* https://mashable.com/article/microsoft-copilot-ai-assistant-announcement Accessed: 2026-03-24.

[69] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to Help With Code Understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 97, 13 pages. doi:10.1145/3597503.3639187

[70] Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, New York, NY, USA, 125–143.

[71] Juliana Nieto-Cardenas, Erin Joy Kramer, Peter Kurto, Ethan Dickey, and Andres Bejarano. 2026. Owlgorithm: Supporting Self-Regulated Learning in Competitive Programming through LLM-Driven Reflection. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1* (USA) *(SIGCSE TS 2026)*. Association for Computing Machinery, New York, NY, USA, 757–763. doi:10.1145/3770762.3772662

[72] Donald A. Norman and James C. Spohrer. 1996. Learner-centered education. *Commun. ACM* 39, 4 (April 1996), 24–27. doi:10.1145/227210.227215

[73] OpenAI. 2022. Introducing ChatGPT. https://openai.com/index/chatgpt/.

[74] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] doi:10.48550/arXiv.2303.08774

[75] Matthew J Page, Joanne E McKenzie, Patrick M Bossuyt, Isabelle Boutron, Tammy C Hoffmann, Cynthia D Mulrow, Larissa Shamseer, Jennifer M Tetzlaff, Elie A Akl, Sue E Brennan, Roger Chou, Julie Glanville, Jeremy M Grimshaw, Asbjørn Hróbjartsson, Manoj M Lalu, Tianjing Li, Elizabeth W Loder, Evan Mayo-Wilson, Steve McDonald, Luke A McGuinness, Lesley A Stewart, James Thomas, Andrea C Tricco, Vivian A Welch, Penny Whiting, and David Moher. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ* 372 (2021). doi:10.1136/bmj.n71

[76] Julieto E Perez and Ethel C Ong. 2025. Prompting for Engagement: Using the ICAP Framework to Guide Prompt Design in LLM-Powered Dialogue-Based Tutoring System for Novice Programmers. *Procedia Computer Science* 272 (2025), 113–121. doi:10.1016/j.procs.2025.10.186

[77] Wyatt Petula, Anushcka Joshi, Peggy Tu, Amrutha Somasundar, and Suman Saha. 2026. DeliverC: Teaching Pointers through GenAI-Powered Game-Based Learning. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1* (USA) *(SIGCSE TS 2026)*. Association for Computing Machinery, New York, NY, USA, 838–844. doi:10.1145/3770762.3772509

[78] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '23)*. Association for Computing Machinery, New York, NY, USA, 108–159. doi:10.1145/3623762.3633499

[79] James Prather, Juho Leinonen, Natalie Kiesler, Jamie Gorson Benario, Sam Lau, Stephen MacNeil, Narges Norouzi, Simone Opel, Vee Pettit, Leo Porter, Brent N. Reeves, Jaromir Savelka, David H. Smith, Sven Strickroth, and Daniel Zingaro. 2025. Beyond the Hype: A Comprehensive Review of Current Trends in Generative AI Research, Teaching Practices, and Tools. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education* (Milan, Italy) *(ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 300–338. doi:10.1145/3689187.3709614

[80] James Prather, Brent N. Reeves, Paul Denny, Brett A. Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *ACM Trans. Comput.-Hum. Interact.* 31, 1, Article 4 (Nov. 2023), 31 pages. doi:10.1145/3617367

[81] James Prather, Brent N Reeves, Juho Leinonen, Stephen MacNeil, Arisoa S Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. 2024. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1 (ICER '24)*. Association for Computing Machinery, New York, NY, USA, 469–486. doi:10.1145/3632620.3671116

[82] Ashwin Rachha and Mohammed Seyam. 2024. LLM-Enhanced Learning Environments for CS: Exploring Data Structures and Algorithms with Gurukul. In *2024 IEEE Frontiers in Education Conference (FIE)*. 1–9. doi:10.1109/FIE61694.2024.10893211

[83] Jake Renzella, Alexandra Vassar, Lorenzo Lee Solano, and Andrew Taylor. 2025. Compiler-Integrated, Conversational AI for Debugging CS1 Programs. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1* (Pittsburgh, PA, USA) *(SIGCSETS 2025)*. Association for Computing Machinery, New York, NY, USA, 994–1000. doi:10.1145/3641554.3701827

[84] Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-Step Hint Generation for Introductory Programming Using Large Language Models. In *Proceedings of the 26th Australasian Computing Education Conference* (Sydney, NSW, Australia) *(ACE '24)*. Association for Computing Machinery, New York, NY, USA, 144–153. doi:10.1145/3636243.3636259

[85] Jeeho Ryoo, Michael Pin-Chuan Lin, Sahil Rai, Wenhao He, Seong Min Park, and Marco Ho. 2025. WIP: Multi-Agent Artificial Intelligence Model to Enhance Self-Regulated Learning and Conceptual Understanding in Computer Science Education. In *2025 IEEE Frontiers in Education Conference (FIE)*. 1–5. doi:10.1109/FIE63693.2025.11328198

[86] Andrey Shcherbakov, Dennis Biström, Truong An Pham, and Leonardo Espinosa-Leal. 2026. Programming Education with LLMs and NPCs: A Dialogical Learning Framework for VS Code. In *Smart Technologies for an All-Electric Society*, Michael E. Auer, Reinhard Langmann, Dominik May, and Manuel Morales (Eds.). Springer Nature Switzerland, Cham, 217–228. doi:10.1007/978-3-032-07316-7_19

[87] Elliot Soloway, Shari L. Jackson, Jonathan Klein, Chris Quintana, James Reed, Jeff Spitulnik, Steven J. Stratford, Scott Studer, Jim Eng, and Nancy Scala. 1996. Learning theory in practice: case studies of learner-centered design. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, British Columbia, Canada) *(CHI '96)*. Association for Computing Machinery, New York, NY, USA, 189–196. doi:10.1145/238386.238476

[88] Raja Sooriamurthi, Xiaoying Tu, and Allison E. Connell Pensky. 2025. A Generative AI Tool to Foster and Assess Authentic Learning: A Case Study in Teaching SQL. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1* (Nijmegen, Netherlands) *(ITiCSE 2025)*. Association for Computing Machinery, New York, NY, USA, 465–471. doi:10.1145/3724363.3729022

[89] SpringerLink. 2026. SpringerLink. https://link.springer.com/. Accessed: 2026-03-24.

[90] Liam Stienstra, Abdallah Mohamed, and Mostafa Mohamed. 2025. Exploring GenAI as a Tutoring Tool: A Case Study in First-Year Computer Programming. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1* (Nijmegen, Netherlands) *(ITiCSE 2025)*. Association for Computing Machinery, New York, NY, USA, 452–457. doi:10.1145/3724363.3729060

[91] John Sweller. 2011. Cognitive Load Theory. In *Cognition in Education*, Jose P. Mestre and Brian H. Ross (Eds.). Psychology of Learning and Motivation, Vol. 55. Academic Press, San Diego, CA, USA, 37–76. doi:10.1016/B978-0-12-387691-1.00002-8

[92] Kay Tang, Hoanh Nguyen Hosea Tong-Ho, Kaitlin Riegel, Paul Denny, Nasser Giacaman, and Juho Leinonen. 2026. Ambiguity by Design: Practicing Requirement Clarification through Natural-Language Dialogue with LLMs. In *Proceedings of the 28th Australasian Computing Education Conference (ACE '26)*. Association for Computing Machinery, New York, NY, USA, 138–147. doi:10.1145/3786228.3786249

[93] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. 2024. The Metacognitive Demands and Opportunities of Generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 680, 24 pages. doi:10.1145/3613904.3642902

[94] Andrew Taylor, Alexandra Vassar, Jake Renzella, and Hammond Pearce. 2024. dcc –help: Transforming the Role of the Compiler by Generating Context-Aware Error Explanations with Large Language Models. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1* (Portland, OR, USA) *(SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 1314–1320. doi:10.1145/3626252.3630822

[95] Karena Tran, Ge Gao, Angela Lombard, Tyler Yu, Haoning Jiang, and Thomas Y. Yeh. 2026. Pacing for Mastery: Optimizing LLM Interactions for Learning. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1* (USA) *(SIGCSE TS 2026)*. Association for Computing Machinery, New York, NY, USA, 1068–1074. doi:10.1145/3770762.3772501

[96] Janneke Van de Pol, Monique Volman, and Jos Beishuizen. 2010. Scaffolding in teacher–student interaction: A decade of research. *Educational psychology review* 22, 3 (2010), 271–296. doi:10.1007/s10648-010-9127-6

[97] L. S. Vygotsky. 1978. *Mind in Society*. Harvard University Press, London, England. doi:10.2307/j.ctvjf9vz4

[98] Haoming Wang, Chengliang Wang, Zhan Chen, Fa Liu, Chunjia Bao, and Xianlong Xu. 2025. Impact of AI-agent-supported collaborative learning on the learning outcomes of University programming courses. *Education and Information Technologies* 30, 12 (2025), 17717–17749. doi:10.1007/s10639-025-13487-8

[99] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. doi:10.52202/068431-1800

[100] David Wood, Jerome S Bruner, and Gail Ross. 1976. The role of tutoring in problem solving. *Journal of child psychology and psychiatry* 17, 2 (1976), 89–100. doi:10.1111/j.1469-7610.1976.tb00381.x

[101] Jinmiao Wu, Thomas Selig, and Erick Purwanto. 2025. TraceMate: Collaborating with AI in Test-Driven Programming . In *2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE Computer Society, Los Alamitos, CA, USA, 253–259. doi:10.1109/VL-HCC65237.2025.00035

[102] Wenyi Xie, Haojie Shi, Yixuan Li, and Junwei Liu. 2025. STAP: A Socratic Tutor for Adaptive Programming with Pedagogical Scaffolding. In *Proceedings of the 2025 2nd International Symposium on Artificial Intelligence for Education*

(ISAIE '25). Association for Computing Machinery, New York, NY, USA, 584–587. doi:10.1145/3775073.3775165

[103] Eric Zabala and Husnu S. Narman. 2024. Development and Evaluation of an AI-Enhanced Python Programming Education System. In *2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*. 787–792. doi:10.1109/UEMCON62879.2024.10754661

[104] Cynthia Zastudil, Magdalena Rogalska, Christine Kapp, Jennifer Vaughn, and Stephen MacNeil. 2023. Generative AI in Computing Education: Perspectives of Students and Instructors. In *2023 IEEE Frontiers in Education Conference (FIE)*. 1–9. doi:10.1109/FIE58773.2023.10343467

[105] Gefei Zhang, Guodao Sun, Meng Xia, and Ronghua Liang. 2026. ClassAid: A Real-time Instructor-AI-Student Orchestration System for Classroom Programming Activities. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 1580, 27 pages. doi:10.1145/3772318.3790824

[106] Yuzhe Zhou, Prithvi Manjunatha Babu, Udayan Pandey, Alejandra J. Magana, and Tianyi Li. 2026. ExPeerience: Towards AI-Assisted Learnersourcing to Bridge Conceptual Understanding and Problem Solving in Database Programming Education. In *Proceedings of the 31st International Conference on Intelligent User Interfaces (IUI '26)*. Association for Computing Machinery, New York, NY, USA, 1178–1201. doi:10.1145/3742413.3789139

[107] Elaine Zhu, Smaran Teja, Chris Coombes, and Daniel Patterson. 2025. FEEDBOT: Formative Design Feedback on Programming Assignments. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1* (Nijmegen, Netherlands) *(ITiCSE 2025)*. Association for Computing Machinery, New York, NY, USA, 576–582. doi:10.1145/3724363.3729063